Modern Python Project Management for Researchers with uv.

Elio Balestrieri

16. Juli 2026

Who am I

- Born in Genoa 🇮🇹
- Grew up in San Marino 🇸🇲
- PhD & Postdoc in Neuroscience 🧠 at University of Münster 🇩🇪
- HPC System Admin 🤖 at University of Münster

Who am I

- Born in Genoa 🇮🇹
- Grew up in San Marino 🇸🇲
- PhD & Postdoc in Neuroscience 🧠 at University of Münster 🇩🇪
- HPC System Admin 🤖 at University of Münster

Enthusiastic about many things... Relevant here: **good**, **open scientific software** and **reproducibility** of scientific workflows.

Who are you?

Who are you?

Scientist 🧑‍🔬

Who are you?

Developer/System Administrator 👨‍💻

Who are you?

Curious 🧙

What is uv?

An extremely fast package and project manager, written in Rust

🤔 And why should I care?

🤔 And why should I care?

- Extremely fast
- Very versatile for both new and existing projects
- It feels like “this is how it was supposed to be done”

OK, where’s the catch?

OK, where's the catch?

There is one, depending on your standpoint on Big Tech:

As of March 19th 2026, **Astral** (the company behind uv) **joined OpenAI**

https://astral.sh/blog/openai

🤷 Naja, what can we do about it?

Not much… Decide whether to use it or not, I guess.

🤷 Naja, what can we do about it?

Not much... Decide whether to use it or not, I guess.

And if you still want to do that, then let's start!

```shell
git clone https://github.com/ElioBalestrieri/python-uv-workshop.git
```

Installation

https://docs.astral.sh/uv/#installation

 & :

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```
Shell

:

```
PS> powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"
```
PowerShell

***Apologetic note**: I have stopped using windows back in 2010* 😅

Installation

https://docs.astral.sh/uv/#installation

 & :

```shell
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

:

```powershell
PS> powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"
```

***Apologetic note**: I have stopped using windows back in 2010* 😅

How is everybody doing?

About virtual environments

By ✋: How many of you are familiar with the concept of virtual environment?

A virtual environment is:

- Contained in a directory (e.g. `.venv`), and consists of a specific Python interpreter and software libraries and binaries which are needed to support a project.
- **Isolated** from **other virtual environments** and Python interpreters and libraries installed system-wide.
- **Not** checked into source control systems such as **Git**.
- Considered as **disposable**.
- **Not** considered as **movable** or **copyable**.

`uv venv / pip`

You can create a new environment, with a specific python version as follows:

```shell
uv venv --python 3.13 my-test-venv
```

Defaults:

- no virtual env name ⊕ `.venv`
- if python version unavailable ⊕ `uv` downloads it for you
- no python version specified ⊕ system default

`uv venv / pip`

You can activate the new environment as follows:

 & :

```Shell
source .venv/bin/activate
```

:

```PowerShell
.venv\Scripts\activate
```

uv venv / pip

You can install new packages in your environment using pip:

```shell
uv pip install numpy pandas
```

uv venv / pip

You can install new packages in your environment using pip:

```shell
uv pip install numpy pandas
```

And you can deactivate the virtual environment with:

```shell
deactivate
```

uv venv / pip

To summarize:

```shell
uv venv --python 3.13 my-test-venv
source my-test-venv/bin/activate
uv pip install numpy pandas
deactivate
```

Your turn 👉

Create a virtual environment with `uv venv` and install your favorite packages with `uv pip`

Stuck? Check: https://docs.astral.sh/uv/pip/

🆒😒 Why do we even need `uv` for this?

Is this really only about having a fast `pip`?

Projects in uv

uv allows the creation of Python projects as organized development environments... Quite in a smooth way:

```shell
uv init my-amazing-project
cd my-amazing project
```

or, alternatively:

```shell
mkdir my-amazing-project
cd my-amazing project
init
```

Projects in `uv`

Already some interesting things:

- by default under version control with **git**
- `pyproject.toml` ⊖ metadata and dependencies.

Projects in `uv`

```shell
uv run main.py
```

Shell

Adds some items in the project:

- `.venv/`
- `uv.lock` a cross platform lockfile containing the exact info about your project dependencies. **Do not edit**

Projects in `uv`

To add new dependencies:

```shell
uv add numpy
```

Or alternatively, edit the dependencies list of `pyproject.toml` and run:

```shell
uv run main.py
```

Projects in uv

`uv add` is also compatible with requirements.txt

```shell
uv add -r requirements.txt
```

Projects in uv

🧑 *when I was young I was simply executing my scripts with python*

OK, fine. Edit your `pyproject.toml` and then:

```shell
uv sync
source .venv/bin/activate
python main.py
```

Your turn 👉

- create a new project
- migrate the dependencies from an old project

  💡 `pip freeze > requirements.txt`
- test whether the imports work

Help yourself with these slides or:

https://docs.astral.sh/uv/guides/projects/#managing-dependencies

Your turn 👉

- create a new project
- migrate the dependencies from an old project

  💡 `pip freeze > requirements.txt`
- test whether the imports work

Help yourself with these slides or:

https://docs.astral.sh/uv/guides/projects/#managing-dependencies

🔄 **alternatively,** if you don't have a project to work with:

`python-uv-workshop/exercises/migrate-project`

Shipping your project

It can be as immediate[1] as:

```shell
uv build
uv publish --token pypi-SeCR3tT0K3N --publish-url https://test.pypi.org/legacy/
```

For testing your upload on test-pypi. You can even omit the `--publish-url` when you are ready to publish to PyPI.

[1] if you don't have C code 😇

Single executable scripts

Although it is *preferable* to work within a project, you can also declare dependencies as **inline metadata** and run the script as a standalone.

For example:

```shell
uv init --script my-standalone-script.py --python 3.13
uv add --script my-standalone-script.py 'numpy' 'pandas' 'matplotlib'
```

Your turn 👉

Scenario
You have one python script that does not require a whole project by itself, and you want to send it around as a standalone.

How can you do that with `uv`?

🔄 **alternatively,** you can create a standalone script from `analysis.py` in `python-uv-workshop/exercises/migrate-project`.

Stuck? Check: https://docs.astral.sh/uv/guides/scripts/#declaring-script-dependencies

Useful links

https://docs.astral.sh/uv/
https://pydevtools.com/
https://www.reddit.com/r/learnpython/comments/1bmxe6i/whats_the_difference_between_pyprojecttoml/

Fin.

Excursus: `pyproject.toml` vs `requirements.txt`

https://pydevtools.com/handbook/explanation/pyproject-vs-requirements/

`requirements.txt` lists packages to install. `pyproject.toml` defines a project. That distinction shapes how dependencies get managed and how projects scale.

Moreover `requirements.txt` has some gaps:

- It demands manual coordination (`pip-compile` or `pip freeze`)
- No Python version constraint.
- No project metadata.

Excursus: `uv tools`

Seem cool, but tbh I have never used them
https://docs.astral.sh/uv/guides/tools/